

Data Warehouse de Notificações de COVID-19 do Espírito Santo: Modelagem Dimensional, ETL e Visualização Analítica

Lorena

22 de junho de 2026

Resumo

Este relatório descreve o desenvolvimento de um Data Warehouse (DW) para análise de notificações de COVID-19 no estado do Espírito Santo, construído sobre PostgreSQL com modelagem dimensional em esquema Floco de Neve (*Snowflake*). O trabalho abrange desde a análise exploratória dos microdados brutos até a construção de um Data Mart pré-agregado, uma rotina de controle de qualidade de carga (*Quality Gate*) e um painel analítico interativo desenvolvido em Streamlit. São discutidas as decisões de modelagem, a estratégia de carga otimizada via COPY, os desafios encontrados durante a implementação do pipeline de ETL e os resultados obtidos.

Sumário

1	Introdução	3
2	Objetivos	3
3	Arquitetura e Tecnologias	3
4	Modelagem Dimensional	4
4.1	Escolha do esquema: Floco de Neve	4
4.2	Comparativo: Estrela vs. Floco de Neve	4
5	Pipeline de ETL	5
5.1	Conversão CSV → Parquet	5
5.2	Carga via COPY	5
5.3	Ordem de execução do pipeline	6
6	Data Mart e Avaliação de Performance	6
7	Controle de Qualidade de Dados (<i>Quality Gate</i>)	7
8	Extensão para Histórico: SCD Tipo 2	7
9	Painel Analítico (Dashboard)	8
10	Conclusão	8
	Referências	9

1 Introdução

O monitoramento epidemiológico de doenças de notificação compulsória, como a COVID-19, gera grandes volumes de dados operacionais que, em seu formato bruto, não são adequados para análises gerenciais e estratégicas. Este projeto tem como objetivo transformar os microdados de notificação de COVID-19 do Espírito Santo em uma estrutura analítica otimizada, seguindo os princípios clássicos de modelagem dimensional (*Kimball*), permitindo consultas rápidas, históricas e agregadas sobre confirmações, óbitos e internações.

2 Objetivos

- Realizar análise exploratória dos dados brutos (CSV de microdados), quantificando valores nulos por coluna;
- Projetar e implementar um modelo dimensional em esquema Floco de Neve para o Data Warehouse `dw_covid`;
- Construir um pipeline de ETL (*Extract, Transform, Load*) eficiente para cargas de grande volume (milhões de registros);
- Criar um Data Mart pré-agregado e avaliar o ganho de performance frente a consultas ad-hoc na tabela fato;
- Desenvolver um painel analítico interativo para visualização dos principais indicadores epidemiológicos;
- Implementar uma rotina de controle de qualidade de dados (*Quality Gate*) anterior à carga da tabela fato;
- Discutir uma extensão do modelo para rastreamento histórico de atributos mutáveis (*Slowly Changing Dimension* Tipo 2).

3 Arquitetura e Tecnologias

O projeto foi implementado utilizando a seguinte pilha tecnológica:

- **PostgreSQL** — sistema gerenciador de banco de dados relacional, hospedando o banco `dw_covid`;
- **Python** (`pandas`, `PyArrow`, `SQLAlchemy`, `psycopg2`) — orquestração do pipeline de ETL e manipulação dos dados;
- **Apache Parquet** — formato de armazenamento colunar intermediário, utilizado para reduzir o consumo de memória durante a leitura do arquivo CSV original;
- **Jupyter Notebook** — ambiente de desenvolvimento e documentação dos exercícios;
- **Streamlit** — biblioteca para construção do painel analítico interativo;
- **Matplotlib / Seaborn** — geração das visualizações estatísticas.

4 Modelagem Dimensional

4.1 Escolha do esquema: Floco de Neve

Optou-se por normalizar a dimensão de localidade em três níveis hierárquicos ($\text{DIM_REGIAO} \rightarrow \text{DIM_MUNICIPIO} \rightarrow \text{DIM_BAIRRO}$), em vez de uma única dimensão desnormalizada (DIM_LOCALIDADE), conforme o modelo estrela tradicional. A Figura 1 ilustra a estrutura resultante.

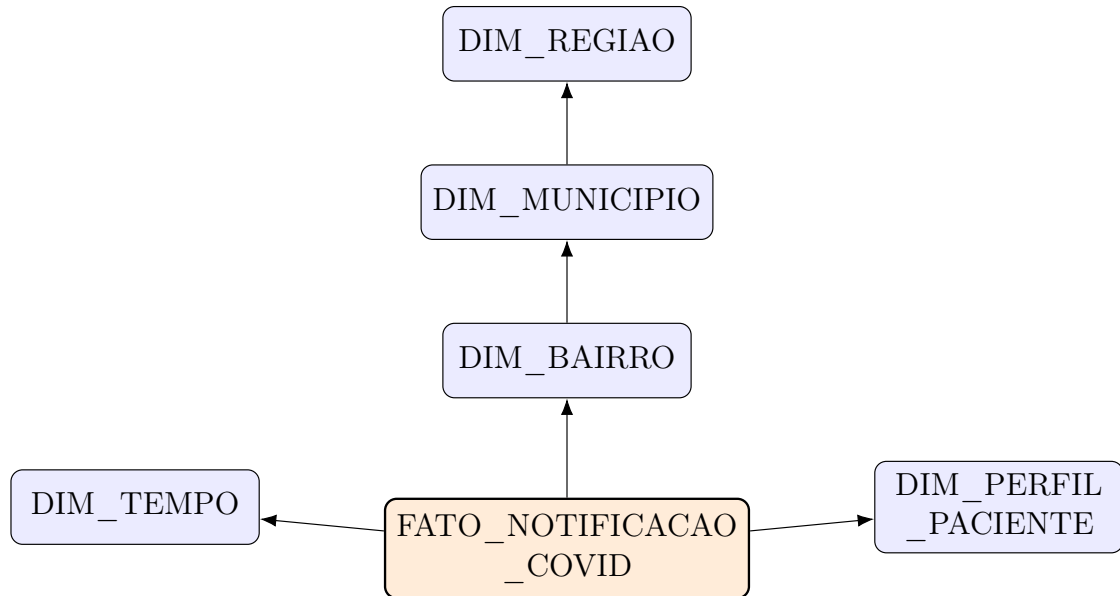


Figura 1: Estrutura simplificada do modelo Floco de Neve implementado.

Além das dimensões geográficas e temporais, o modelo inclui dimensões de perfil clínico (DIM_CLASSIFICACAO , $\text{DIM_PERFIL_PACIENTE}$) e duas dimensões de detrito (*junk dimensions*): DIM_SINTOMAS e DIM_COMORBIDADE , que agrupam atributos binários de baixa cardinalidade para evitar a explosão do número de colunas na tabela fato.

4.2 Comparativo: Estrela vs. Floco de Neve

A Tabela 1 resume os principais trade-offs entre as duas abordagens, considerando o contexto específico deste projeto.

Tabela 1: Comparativo entre modelo Estrela e modelo Floco de Neve para a dimensão de localidade.

Critério	Estrela (DIM_LOCALIDADE única)	Floco de Neve (3 tabelas)
Redundância	Alta — município repetido em cada bairro	Baixa — município armazenado uma única vez
Performance de leitura	Melhor — menos <i>joins</i>	Pior — exige junção adicional (bairro → município → região)
Manutenção	Mais difícil — correções exigem UPDATE em múltiplas linhas	Mais fácil — correção em uma única linha
Integridade	Risco de inconsistência textual	Garantida por chave estrangeira
Espaço em disco	Maior	Menor
Complexidade de ETL	Mais simples	Mais complexa (ordem de carga hierárquica)

Concluiu-se que, para este projeto, o ganho de manutenção do modelo Floco de Neve compensa o custo adicional de junção, especialmente porque o Data Mart (Seção 6) absorve esse custo ao pré-agregar os dados.

5 Pipeline de ETL

5.1 Conversão CSV → Parquet

O arquivo de microdados original, em formato CSV de grande volume, foi convertido para Apache Parquet por meio de leitura em blocos (*chunks*) de 100.000 linhas, evitando o carregamento integral do arquivo em memória:

Listing 1: Conversão incremental de CSV para Parquet.

```

1 for chunk in pd.read_csv(path_csv, sep=';', encoding='LATIN1',
2                           low_memory=False, chunksize=100000):
3     table = pa.Table.from_pandas(chunk, preserve_index=False)
4     if writer is None:
5         writer = pq.ParquetWriter(path_parquet, table.schema,
6                                   compression='snappy')
7     writer.write_table(table)
8     del chunk, table
9     gc.collect()

```

5.2 Carga via COPY

A carga inicial dos dados brutos para o schema de *staging* (`stg.notificacao_raw`) utiliza o comando `COPY` nativo do PostgreSQL, em substituição ao método tradicional `INSERT` linha a linha do `pandas.to_sql`. Essa mudança foi motivada por ganhos de desempenho de até uma ordem de magnitude em cargas de milhões de registros:

Listing 2: Carga em massa via COPY.

```
1 buffer = io.StringIO()
2 df.to_csv(buffer, index=False, header=False, sep=',', na_rep='')
3 buffer.seek(0)
4
5 with engine.connect() as conn:
6     cursor = conn.connection.cursor()
7     cursor.copy_expert(
8         f"COPY {schema}.{tabela} FROM STDIN WITH "
9         f"(FORMAT csv, DELIMITER ',', QUOTE '\"', NULL '')",
10        buffer
11    )
12    conn.connection.commit()
```

5.3 Ordem de execução do pipeline

Um dos principais desafios encontrados durante o desenvolvimento foi a dependência de ordem entre as etapas do pipeline: o script de ETL que popula dimensões e fato consulta

diretamente a tabela de *staging* (`stg.notificacao_raw`); caso essa tabela ainda esteja vazia no momento da execução do ETL, as dimensões resultam praticamente

vazias (apenas com a linha de valor ausente "*Desconhecido*"). A solução adotada foi separar o script SQL original em dois arquivos distintos — um de definição de estrutura

(*DDL*) e outro de carga (*ETL*) — e fixar a seguinte ordem de execução:

1. Recriação do banco de dados (`DROP/CREATE DATABASE`);
2. Criação de schemas e tabelas (`snowflake_ddl.sql`);
3. Carga dos dados brutos em `stg.notificacao_raw` via `COPY`;
4. Execução do ETL de população de dimensões e fato (`snowflake_etl.sql`);
5. Criação do Data Mart;
6. Execução do *Quality Gate*.

6 Data Mart e Avaliação de Performance

Foi criada uma *Materialized View* no schema `mart`

(`mart.mv_resumo_municipio_mes`), pré-agregando o total de notificações confirmadas, óbitos e internações por município e mês. A comparação de desempenho entre

a consulta direta na tabela fato e a consulta sobre a Materialized View foi realizada com

`EXPLAIN ANALYZE`, evidenciando a redução do tempo de execução ao evitar os *joins* repetidos entre fato e dimensões a cada consulta analítica repetitiva.

Listing 3: Criação do Data Mart pré-agregado.

```
1 CREATE MATERIALIZED VIEW mart.mv_resumo_municipio_mes AS
2 SELECT
3     m.municipio ,
4     t.ano_mes , t.ano , t.mes ,
5     SUM(f.flag_confirmado) AS total_confirmados ,
6     SUM(f.flag_obito_covid) AS total_obitos ,
7     SUM(f.flag_internado) AS total_internacoes ,
8     SUM(f.qtd_notificacao) AS total_notificacoes
9 FROM dw.fato_notificacao_covid f
10 JOIN dw.dim_bairro b ON b.sk_bairro = f.sk_bairro
11 JOIN dw.dim_municipio m ON m.sk_municipio = b.fk_municipio
12 JOIN dw.dim_tempo t ON t.sk_tempo = f.sk_data_notificacao
13 GROUP BY m.municipio , t.ano_mes , t.ano , t.mes
14 WITH DATA;
```

7 Controle de Qualidade de Dados (*Quality Gate*)

Antes de cada carga da tabela fato, é executada uma *stored procedure* de validação que verifica três condições:

1. **Existência da linha "Desconhecido"** em todas as dimensões, garantindo que registros com atributos ausentes ou não mapeados possam ser referenciados sem violar integridade referencial;
2. **Ausência de chaves substitutas órfãs** na tabela fato — nenhuma chave estrangeira deve apontar para uma linha inexistente em sua dimensão correspondente;
3. **Conformidade de volume** — a soma de `qtd_notificacao` na fato deve corresponder à contagem de registros (`COUNT(*)`) da tabela de *staging*, evitando perdas silenciosas de dados durante o ETL.

Caso qualquer uma das validações falhe, a carga é interrompida e um erro detalhado é reportado, prevenindo a publicação de dados inconsistentes no Data Warehouse.

8 Extensão para Histórico: SCD Tipo 2

Para acomodar um atributo potencialmente mutável no tempo (por exemplo, a população estimada de um município), propõe-se a adaptação da dimensão de município para o padrão

Slowly Changing Dimension Tipo 2, introduzindo os campos de controle:

- `data_inicio` — data a partir da qual aquela versão do registro é válida;
- `data_fim` — data em que a versão deixou de ser válida (NULL enquanto vigente);

- `flag_atual` — indicador booleano da versão vigente do registro.

Quando um novo valor de população é identificado para um município já existente, a versão

anterior é *fechada* (atualização de `data_fim` e `flag_atual = false`) e uma nova linha é inserida com nova chave substituta, preservando o histórico completo de alterações e permitindo que fatos antigos continuem referenciando a versão da dimensão vigente à época do evento.

9 Painel Analítico (Dashboard)

O painel de visualização foi implementado em Streamlit, consumindo diretamente as *queries* do Data Warehouse, e contempla quatro indicadores principais:

- Série temporal de notificações;
- Mapa de calor de notificações por município e mês;
- Pirâmide etária dos óbitos;
- Top-5 comorbidades associadas a óbitos.

A aplicação utiliza mecanismos de *cache* (`st.cache_resource` para a conexão e `st.cache_data` para os resultados de consulta), reduzindo a carga sobre o banco de dados durante interações repetidas do usuário com a interface.

10 Conclusão

O desenvolvimento deste Data Warehouse evidenciou, na prática, desafios típicos de projetos de Business Intelligence sobre dados de grande volume: a importância da ordem de

execução em pipelines de ETL com dependências implícitas entre staging e modelo dimensional, o impacto do método de carga (*INSERT* versus *COPY*) sobre o tempo total de processamento, e a necessidade de mecanismos de controle de qualidade antes da publicação de dados analíticos. O modelo Floco de Neve adotado demonstrou-se

adequado ao cenário de baixa frequência de atualização das dimensões geográficas, enquanto o Data Mart pré-agregado mostrou-se eficaz para acelerar consultas analíticas

recorrentes, validando a abordagem clássica de modelagem dimensional de Kimball para este domínio epidemiológico.

Referências

- KIMBALL, R.; ROSS, M. *The Data Warehouse Toolkit*. 3. ed. Wiley, 2013.
- Documentação oficial do PostgreSQL. Disponível em: <https://www.postgresql.org/docs/>.
- Documentação oficial do Streamlit. Disponível em: <https://docs.streamlit.io/>.